IB113 Úvod do programování a algoritmizace

Cvičení 3

Jaromír Plhák

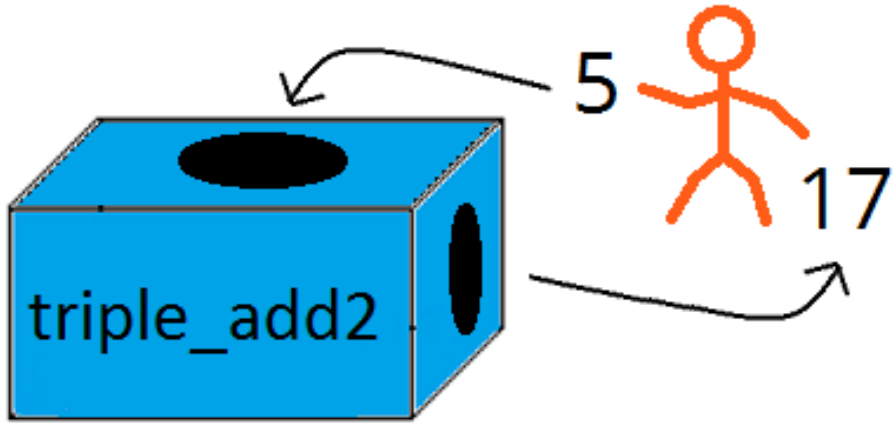




Funkce

- Krabička, která vezme vstup, něco s ním udělá a vrátí výstup

```
def triple_add2 (x: int) -> int:
        return 3 * x + 2
```




```
>>> triple_add2 (5)
17
```

Návratová hodnota

- Funkce **vrátí** výstup, tzv. návratovou hodnotu, volajícímu
- Všimněte si, že příkaz return hodnota okamžitě končí běh funkce a vrací hodnotu volající funkci

```
def triple_add2 (x: int) -> int:
        return 3 * x + 2
```

```
>>> return_value = triple_add2 (5)
>>> return_value
17
```

Procedury a funkce – definice (opakování)

- Klíčové slovo def
- Mezera
- Jméno funkce
- Seznam parametrů v závorkách
  - Žádné parametry - prázdné závorky
- Dvojtečka
- Nový řádek
- Odsazení příkazu funkce

```
def my_funct (par1 , par2 , ...) : # header
        statement_1 # body
        ...
        statement_n
```

Příklad funkce

```
def greater_less_equal_5 (number: int) -> int:
        if number > 5:
                return 1
        elif number < 5:
                return -1
        else :
                return 0

>>> a = greater_less_equal_5(4)
>>> b = greater_less_equal_5(5)
>>> c = greater_less_equal_5(6)
```

print vs. return

```
def triple_add2_return (x: int) -> int:
        return 3 * x + 2

def triple_add2_print (x: int) -> None:
        print (3 * x + 2)


>>> return_value = triple_add2_return(5)
>>> return_value
17
>>> return_value = triple_add2_print(5)
17
>>> return_value
None
```

Lokální platnost proměnných

```
def double(n: int) -> int:
        return 2 * n
def triple(n: int) -> int: # Different n than before
        return 3 * n
def foo (a: int, b: int) -> int:
        return double(a) + triple(b)

>>> foo (5, 7)
>>> double (5) + triple (7)
>>> 10 + triple (7)
>>> 10 + 21
>>> 31
```

Pycodestyle (dříve pep8)

- Style Guide for Python Code
  - Oficiální doporučení
  - https://www.python.org/dev/peps/pep-0008/
- Důležité body z PEP8:
  - Zarovnávání kódu; max. délka řádku 79 znaků; dva prázdné řádky mezi funkcemi
  - Mezery kolem operátorů; žádné mezery za otevírací (před uzavírací) závorkou
    - Výjimka: žádné mezery kolem = u parametrů funkcí
  - Pojmenování proměnných a funkcí: lower_case_with_underscores
  - Konstanty: UPPER_CASE_WITH_UNDERSCORES
  - Třídy (později): CamelCase

Úvodní úkol + Společně řešený úkol

- Napište funkci digit_sum(num), která vrátí ciferný součet čísla num
- Rozšíření v případě dostatku času
  - Napište funkci repeated_digit_sum(num), která opakovaně vypočítá ciferný součet čísla n. Ze získaného ciferného součtu opět vypočítá ciferný součet (pokud se jedná o víceciferný součet) a tento postup opakuje dokud nezbude jednociferné číslo, které vrátí. Ke svému řešení využijte předchozí funkci
  - Vyzkoušejte si debugger pro otestování, že vaše funkce funguje správně

Úkoly

- Pomocí textové grafiky vykreslete vyplněný čtverec o straně 5
  - Velikost 5 je parametr funkce

```
#####
#####
#####
#####
#####
```

Úkoly

- Pomocí textové grafiky vykreslete prázdný čtverec o straně 5
  - Velikost 5 je parametr funkce

```
#####
#...#
#...#
#...#
#...#
#####
```

Úkoly

- Pomocí textové grafiky vykreslete šachovnici o velikosti 8x8
  - Velikost šachovnice je parametrem funkce

```
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
```

Úkoly

- Vypište součet prvních N čísel počínaje jedničkou (pro jedničku vrací 1, pro dvojku 3, pro trojku 6, pro čtyřku 10, …)

Bonusové úkoly

- Převod na binární číslo
  - Parametrem funkce je celé číslo v desítkové soustavě
  - Funkce vrací výsledek v binární soustavě
  - Např. 10 => 1010
- k-tý prvek Fibonacciho posloupnosti
  - Fibonnaciho posloupnoust začíná hodnotami 0, 1
  - Každý další prvek je součtem dvou předchozích
  - Funkce za parametr bere pořadí prvku, který požadujeme.